BSc (Hons) in **Computer Science**

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure
Environment
Actuators

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width and self.height (grid dimensions)
self.walls (obstacle locations)
self.food_positions (food locations)

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Classification: Single-Agent.
Justification: There are no opponent variables (like self.opponents) or other active entities defined in __init__. Only a single agent moves through the grid to collect food.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of everything the agent's sensors have perceived/received from the environment up to the current point in time

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

It represents the Sensors (S) component.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Explanation: The dictionary returned by get_percept() provides complete information about the grid state, including the exact agent_pos, whether food is nearby (smells_food), wall hits (hit_wall), total score, and remaining_food count. The agent does not have hidden variables obscuring the board state.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance measure (P).

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Measuring external results evaluates the agent's actual success and behavior within the physical environment. If performance were measured by internal processing effort, an agent could be rewarded simply for computing complex loops or idling

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?**

It shifts the environment from Fully Observable to Partially Observable because the complete state of the environment contains critical state information (traps) that is obscured from the agent's percepts

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Rationality requires the agent to select actions that maximize its expected score based on its percept sequence. The 'smells_toxin' sensor informs the agent of immediate danger, allowing it to choose moves that avoid penalties (-15 score) and

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

In the Vacuum World, if the performance metric only rewards the agent for picking up dirt, a rational agent could exploit the metric by dumping dirt back onto the floor and cleaning it repeatedly to maximize its score indefinitely, rather than keeping the

Finalize and Lock Lab 01 Analytical Evaluation